Lập trình mạng · Chương 4

Message broker: RabbitMQ, exchange, queue và binding

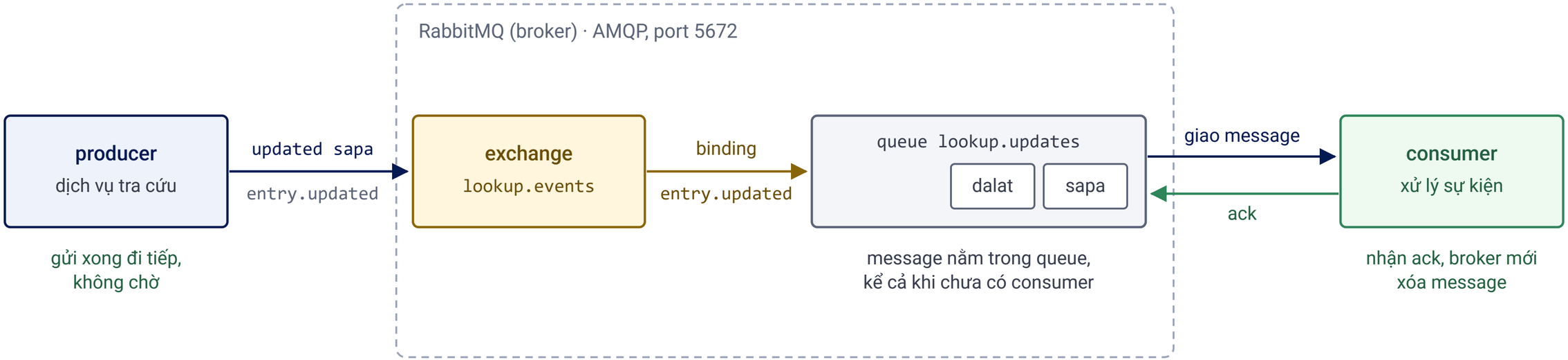
TS. Đặng Ngọc Hùng

Báo sự kiện không cần chờ: giao tiếp bất đồng bộ qua message broker

1. RMI, SOAP, REST, gRPC: bên gọi **chờ** kết quả; bên nhận phải **đang chạy**
2. Báo sự kiện cập nhật: bên gửi không cần kết quả, không nên phụ thuộc trạng thái bên nhận
3. **Message broker**: trung gian nhận, lưu và chuyển message, cho phép **giao tiếp bất đồng bộ**

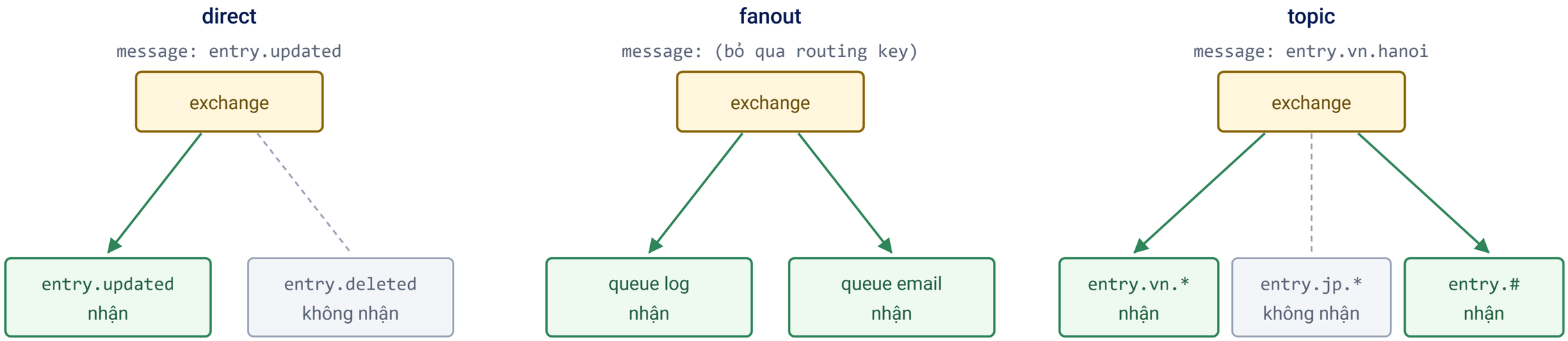
> **Câu hỏi của phần:** message đi qua broker thế nào, và làm sao để message không bị mất?

Producer gửi tới exchange; binding đưa message vào queue; consumer ack



Exchange nhận message, **binding** quyết định **queue** nào nhận; message nằm trong queue đến khi consumer gửi **acknowledgement**.

Ba loại exchange thường dùng: direct, fanout, topic



routing key khớp đúng binding

mọi queue đã bind đều nhận một bản

khớp mẫu: * đúng một từ, # không hoặc nhiều từ

Loại **exchange** quyết định cách chọn **queue**: direct khớp đúng routing key, fanout gửi mọi queue, topic khớp theo mẫu.

Khai báo exchange, queue và binding bằng amqp-client

```Java
channel.exchangeDeclare(EXCHANGE, BuiltinExchangeType.DIRECT);
channel.queueDeclare(QUEUE, true, false, false, null);  // durable: RabbitMQ 4.3 chặn queue thường không durable
channel.queueBind(QUEUE, EXCHANGE, ROUTING_KEY);  // binding: message có routing key này vào queue này
```

- Producer và consumer đều khai báo trước khi dùng: khai báo lại với cùng tham số không đổi gì
- Queue **durable**: RabbitMQ 4.3 không cho tạo queue thường không durable
- Producer chỉ gọi `basicPublish(exchange, routingKey, ...)`, không biết queue nào nhận

Consumer tắt, message vẫn chờ trong queue

Java

```
channel.basicConsume(Topology.QUEUE, autoAck, onMessage, consumerTag -> { });
sleep(200);  // mô phỏng xử lý
if (!autoAck) {
    channel.basicAck(delivery.getEnvelope().getDeliveryTag(), false);  // xử lý xong mới ack
}
```

Kết quả chạy

```
$ UpdatePublisher sapa dalat
published "updated sapa"
published "updated dalat"
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
lookup.updates  2  0
$ UpdateConsumer --max 2
consumer waiting on lookup.updates (manual ack)
got "updated sapa"
got "updated dalat"
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
lookup.updates  0  0
```

autoAck: broker xóa message ngay khi giao, consumer chết là mất

Java

```
channel.basicConsume(Topology.QUEUE, autoAck, onMessage, consumerTag -> { });
Runtime.getRuntime().halt(1);  // tiến trình chết giữa chừng, chưa kịp ack
```

Kết quả chạy

```
$ UpdatePublisher sapa dalat hue
published "updated sapa"
published "updated dalat"
published "updated hue"
$ UpdateConsumer --auto-ack --crash-on dalat
consumer waiting on lookup.updates (autoAck)
got "updated sapa"
got "updated dalat"
crash while processing "updated dalat"
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
lookup.updates  0  0
```

- autoAck: sapa xử lý xong, **dalat và hue mất**, queue còn 0

Ack thủ công sau khi xử lý: message chưa ack được giao lại

Java

```
channel.basicAck(delivery.getEnvelope().getDeliveryTag(), false);  // xử lý xong mới ack
```

Kết quả chạy

```
$ UpdatePublisher sapa dalat hue
published "updated sapa"
published "updated dalat"
published "updated hue"
$ UpdateConsumer --crash-on dalat
consumer waiting on lookup.updates (manual ack)
got "updated sapa"
got "updated dalat"
crash while processing "updated dalat"
$ rabbitmqctl list_queues name messages_ready messages_unacknowledged
name  messages_ready  messages_unacknowledged
lookup.updates  2  0
$ UpdateConsumer --max 2
consumer waiting on lookup.updates (manual ack)
got "updated dalat" (redelivered)
got "updated hue" (redelivered)
```

- Ack thủ công: dalat, hue quay lại queue; consumer sau nhận lại với cờ **redelivered**

Câu hỏi 1

Trong RabbitMQ, một message nằm trong queue đến khi nào?

A. Đến khi producer đóng kết nối.

B. Đến khi consumer nhận và gửi ack cho message đó (hoặc được giao với autoAck).

C. Chỉ trong 1 giây, rồi broker tự xóa.

D. Đến khi exchange chuyển message sang queue khác.

Câu hỏi 1

Trong RabbitMQ, một message nằm trong queue đến khi nào?

A. Đến khi producer đóng kết nối.

B. Đến khi consumer nhận và gửi ack cho message đó (hoặc được giao với autoAck).

C. Chỉ trong 1 giây, rồi broker tự xóa.

D. Đến khi exchange chuyển message sang queue khác.

Broker giữ message trong queue cho tới khi consumer ack. Với autoAck, broker coi message là xong ngay khi giao. Producer đóng kết nối không ảnh hưởng message đã gửi.

Câu hỏi 2

Consumer dùng autoAck và bị lỗi, tiến trình chết khi đang xử lý một message. Message đó ra sao?

A. Quay lại queue và được giao cho consumer khác.

B. Được giao lại cho chính consumer đó khi nó chạy lại.

C. Nằm ở trạng thái chưa ack cho tới khi quản trị viên xử lý.

D. Mất, vì broker đã xóa message ngay khi giao.

Câu hỏi 2

Consumer dùng autoAck và bị lỗi, tiến trình chết khi đang xử lý một message. Message đó ra sao?

A. Quay lại queue và được giao cho consumer khác.

B. Được giao lại cho chính consumer đó khi nó chạy lại.

C. Nằm ở trạng thái chưa ack cho tới khi quản trị viên xử lý.

D. Mất, vì broker đã xóa message ngay khi giao.

Với autoAck, broker xóa message ngay khi giao cho consumer. Consumer chết giữa chừng thì message mất; chỉ ack thủ công sau khi xử lý mới giữ được message.

Tổng kết

1. **Message broker** như **RabbitMQ** (**AMQP**) cho **giao tiếp bất đồng bộ**: **producer** gửi rồi đi tiếp

2. **Exchange** nhận message, **binding** chọn **queue**; message chờ trong queue tới khi **consumer** nhận

3. **Acknowledgement** sau khi xử lý xong: consumer chết thì message được giao lại; autoAck làm mất message

Từ khóa cần nhớ giao tiếp bất đồng bộ · message broker · RabbitMQ · AMQP · producer · consumer · exchange · queue · binding · acknowledgement

Mở rộng

Chủ đề nâng cao

- Durable queue và persistent message
- Dead letter exchange
- Headers exchange: định tuyến theo header của message

Bài tập mở rộng

Gửi message persistent (queue đã durable); khởi động lại RabbitMQ và kiểm tra message nào còn lại trong queue.

Tiếp theo

Bài toán ứng dụng: thông báo bất đồng bộ qua message broker